

3. Práctica de TAD - Listas enlazadas

Índice

Bloque 0 — Preparación: nodos y enlaces · pág. 3

- [Ejercicio 1](#) · pág. 3 — encadenar tres `_Nodo` a mano y recorrerlos con `.prox`.
- [Ejercicio 2](#) · pág. 4 — función `ver_lista(nodo)` : recorrer la cadena con un `while` e imprimir cada dato.
- [Ejercicio 3](#) · pág. 4 — desenganchar un nodo del medio cambiando una sola referencia `prox`.
- [Ejercicio 4](#) · pág. 5 — en papel: predecir qué imprime cada recorrido al reacomodar enlaces y diagramar el estado.
- [Ejercicio 5](#) · pág. 5 — la variable `lista` como puerta de entrada: borrar el primer elemento con `lista = lista.prox`.

Bloque 1 — El esqueleto de `ListaEnlazada` · pág. 7 (incluye la clase completa y sus invariantes)

- [Ejercicio 6](#) · pág. 9 — `__len__` apoyado en `self.len`, sin recorrer los nodos.
- [Ejercicio 7](#) · pág. 10 — `__str__` con formato `[a, b, c]`, más la variante opcional `ver_enlaces()`.
- [Ejercicio 8](#) · pág. 10 — `append(x)` : agregar al final, con `insert` o recorriendo hasta el último nodo.
- [Ejercicio 9](#) · pág. 11 — `index(x)` : posición de la primera aparición, o mensaje de error y `None`.

Bloque 2 — Recorridos que modifican la lista · pág. 12

- [Ejercicio 10](#) · pág. 12 — `marcar_todos(buscado, nuevo)` : reemplazar todas las apariciones en un solo recorrido.
- [Ejercicio 11](#) · pág. 12 — `sacar_menores_que(limite)` : eliminar los menores al límite y devolver cuántos sacó.
- [Ejercicio 12](#) · pág. 13 — depuración: errores de `sacar_repetidos_consecutivos` y `vaciar_hasta`, y qué invariante rompe cada uno.
- [Ejercicio 13](#) · pág. 14 — `concatenar(otra)` : agregar al final los elementos de otra lista sin modificarla.

Bloque 3 — Transformaciones e inversión · pág. 15

- [Ejercicio 14](#) · pág. 15 — `dar_vuelta()` : invertir la lista in-place usando tres referencias simultáneas.
- [Ejercicio 15](#) · pág. 15 — `intercalar(valor, cada_k)` : insertar un valor cada `k` elementos originales, con `k` validado.
- [Ejercicio 16](#) · pág. 16 — función `copiar_desde(lista, objetivo)` : nueva lista desde la primera aparición del objetivo.

Bloque 4 — Lista enlazada con referencia al final · pág. 18

- [Ejercicio 17](#) · pág. 18 — clase `ListaConFinal` con `prim` y `last` : `append` y `pop`, y por qué uno es constante y el otro no.

Bloque 5 — Listas doblemente enlazadas · pág. 20 (presenta `_NodoDoble` y el invariante de los enlaces cruzados)

- [Ejercicio 18](#) · pág. 21 — `ver_al_reves()` recorriendo con `ant`, y `pop()` del último en tiempo constante.
- [Ejercicio 19](#) · pág. 22 — `insert(i, x)` y `pop(i)` en el medio de la lista, con cuatro enlaces en juego.

Bloque 6 — Usar el TAD desde afuera · pág. 24

- [Ejercicio 20](#) · pág. 24 — como cliente: `atender_siguiente` y `adelantar` usando solo la interfaz pública del TAD.

Bloque 0 — Preparación: nodos y enlaces

Este bloque es preparatorio y todavía no usa ningún tipo abstracto de datos. Los ejercicios trabajan directamente con nodos y con las referencias que los encadenan, que son el material sobre el que en el bloque 1 se construye la clase `ListaEnlazada`. Son ejercicios cortos, de un solo concepto cada uno, y conviene resolverlos en orden, porque cada uno reutiliza la cadena de nodos armada en el anterior.

En los diagramas de esta práctica, cada nodo se dibuja como una caja con dos partes: el dato que guarda y su enlace al siguiente. El símbolo `•` representa una referencia a otro nodo, y `/` representa `None` (no hay siguiente).

```
[Ana|•] --> [Bruno|•] --> [Carla|/]
```

Ejercicio 1

Creá tres nodos que representen los turnos de un kiosco, en este orden:

1. `"Ana"`
2. `"Bruno"`
3. `"Carla"`

Usá la clase `_Nodo` de abajo. El nodo de Ana debe apuntar al de Bruno, y el de Bruno al de Carla. Carla no apunta a nadie (`prox` es `None`).

```
from typing import Any

class _Nodo:
    def __init__(self, dato: Any = None, prox=None):
        self.dato = dato
        self.prox = prox

    def __str__(self):
        return str(self.dato)
```

La cadena que tenés que armar es esta:

```
n_ana      n_bruno      n_carla
[Ana|•] ----> [Bruno|•] ----> [Carla|/]
```

Ejemplo de uso esperado (los nombres de las variables pueden variar):

```
>>> n_ana = _Nodo("Ana")
>>> n_bruno = _Nodo("Bruno")
>>> n_carla = _Nodo("Carla")
>>> n_ana.prox = n_bruno
```

```
>>> n_bruno.prox = n_carla
>>> str(n_ana)
'Ana'
>>> str(n_ana.prox)
'Bruno'
>>> str(n_ana.prox.prox)
'Carla'
>>> n_ana.prox.prox.prox is None
True
```

Ejercicio 2

Escribí la función `ver_lista(nodo: _Nodo | None) -> None` que recorra todos los nodos a través de sus enlaces e imprima el contenido de cada uno, uno por línea.

Usá un `while`. No uses listas nativas de Python para armar el recorrido.

```
>>> # Continuación del ejercicio 1
>>> ver_lista(n_ana)
Ana
Bruno
Carla
>>> ver_lista(None) # lista vacía: no imprime nada
```

Ejercicio 3

Partiendo de la cadena Ana → Bruno → Carla del ejercicio 1, “desenganchá” a Bruno: hacé que Ana apunte directo a Carla, sin borrar el nodo de Bruno de la memoria (solo sacalo de la cadena).

Después, mostrará con `ver_lista` que la cadena quedó Ana → Carla.

Antes:

n_ana	n_bruno	n_carla
[Ana •]	----> [Bruno •]	----> [Carla /]

Después:

n_ana	n_bruno	n_carla
[Ana •]	[Bruno •] ----> [Carla /]	
		^
+-----+		

```
>>> n_ana.prox = n_carla
>>> ver_lista(n_ana)
Ana
Carla
```

En el segundo diagrama, la flecha larga que baja desde el enlace de Ana y sube hasta Carla es el nuevo valor de `n_ana.prox`. Fijate que el nodo de Bruno quedó intacto: sigue existiendo y su propia flecha sigue apuntando a Carla. Lo único que cambió es que ya no se llega a Bruno arrancando desde Ana.

Explicá en una o dos oraciones por qué alcanza con cambiar una sola referencia `prox` para sacar un nodo del medio (pensá en cómo están guardados los nodos en memoria, a diferencia de un arreglo contiguo).

Ejercicio 4

Este ejercicio se resuelve **en papel, sin ejecutar el código**. Después, si querés, lo corrés para verificar.

Dado el siguiente código, que usa `_Nodo` y la función `ver_lista` del ejercicio 2:

```
p = _Nodo(10)
q = _Nodo(20)
r = _Nodo(30)
p.prox = q
q.prox = r

ver_lista(p)      # (a)

q.prox = None
ver_lista(p)      # (b)

q.prox = r
p.prox = r
ver_lista(p)      # (c)
ver_lista(q)      # (d)
```

Respondé:

1. ¿Qué imprime cada una de las cuatro llamadas, `(a)`, `(b)`, `(c)` y `(d)`?
2. Dibujá el estado de los tres nodos y de sus enlaces justo antes de la llamada `(c)`, usando la notación de cajas y flechas del principio del bloque.
3. En `(c)` y `(d)` se recorre la misma colección de nodos pero se imprimen cosas distintas. Explicá por qué en una o dos oraciones.

Ejercicio 5

En los ejercicios anteriores, para llegar a la cadena usabas siempre la variable `n_ana`. Eso trae un problema: si querés borrar el primer elemento, no te alcanza con reacomodar enlaces, porque el nodo que hay que sacar es justamente aquel por el que entrás a la lista.

La solución es tener una variable aparte, que llamaremos `lista`, cuyo único trabajo es referenciar al primer nodo, sea cual sea. Cuando el primero cambia, se actualiza `lista`.

Armá de nuevo la cadena Ana → Bruno → Carla, agregá la variable `lista` y después:

1. Borrá el primer elemento de la lista **sin perder los otros dos**. La única línea que necesitás es `lista = lista.prox`.
2. Mostrá con `ver_lista(lista)` que quedaron Bruno y Carla.
3. Borrá también a Bruno de la misma forma y mostrá que queda solo Carla.

Antes:
lista
|
v
[Ana|•] ----> [Bruno|•] ----> [Carla|/]

Después de lista = lista.prox:
lista
|
v
[Ana|•] ----> [Bruno|•] ----> [Carla|/]

```
>>> n_carla = _Nodo("Carla")
>>> n_bruno = _Nodo("Bruno", n_carla)
>>> n_ana = _Nodo("Ana", n_bruno)
>>> lista = n_ana
>>> ver_lista(lista)
Ana
Bruno
Carla
>>> lista = lista.prox
>>> ver_lista(lista)
Bruno
Carla
>>> lista = lista.prox
>>> ver_lista(lista)
Carla
```

Fijate en el diagrama que el nodo de Ana no cambió en nada: sigue guardando "Ana" y sigue apuntando a Bruno. Lo único que se movió es la variable lista .

Para cerrar, respondé: sacar el primer elemento se resuelve con una sola asignación, sin recorrer nada. Sacar un elemento del medio, en cambio, obliga a llegar caminando hasta el nodo anterior. ¿Cuál de las dos operaciones tarda lo mismo sin importar el largo de la lista, y cuál tarda más a medida que la lista crece?

Bloque 1 — El esqueleto de `ListaEnlazada`

En los ejercicios anteriores manipulaste los enlaces en forma directa, y eso tiene tres costos concretos: hay que llevar la cuenta de qué variable referencia a cada nodo, el primer elemento requiere un tratamiento distinto del resto, y una asignación incorrecta deja la cadena inconsistente sin producir ningún error inmediato.

La clase `ListaEnlazada` separa dos responsabilidades que hasta ahora estaban mezcladas. Por dentro guarda los mismos nodos y hace las mismas asignaciones sobre `prox`. Por fuera expone un conjunto de operaciones con nombre (`append`, `insert`, `pop`, `remove`, `index`) y garantiza dos condiciones: que `prim` referencie siempre al primer nodo, que es el papel que cumplía la variable `lista` del ejercicio 5, y que `len` contenga la cantidad real de elementos. Quien usa la clase **como cliente** opera solamente a través de esas operaciones, sin acceder a los enlaces.

El guión bajo en `_Nodo` es una convención de Python para señalar que un nombre pertenece a la implementación interna. Indica que los nodos son un detalle de `ListaEnlazada`: el lenguaje no impide usarlos desde afuera, pero hacerlo anula la separación que la clase establece.

Métodos y funciones. De acá en adelante vas a escribir código de dos formas distintas, y conviene saber de entrada cuál corresponde en cada caso. Las operaciones que forman parte del TAD se escriben como **métodos** de la clase: reciben `self`, acceden a `prim`, a `len` y a los enlaces, y son responsables de mantener los invariantes. El código que resuelve un problema **usando** el TAD se escribe como **función** por fuera de la clase, y se limita a llamar a esas operaciones. La función `ver_lista` del ejercicio 2 es un caso aparte: en el bloque 0 no existía todavía ninguna clase, así que un método no era una opción.

En la clase de abajo, `insert`, `pop` y `remove` ya están implementados. No es necesario analizarlos línea por línea todavía: para los ejercicios de este bloque alcanza con conocer su comportamiento. Conviene volver sobre ellos en el bloque 2, donde vas a escribir recorridos con la misma estructura.

```
from typing import Any

class _Nodo:
    def __init__(self, dato: Any = None, prox=None):
        self.dato = dato
        self.prox = prox

    def __str__(self):
        return str(self.dato)

class ListaEnlazada:
    """Modela una lista enlazada."""

    def __init__(self) -> None:
```

```

    """Crea una lista enlazada vacía."""
    # Referencia al primer nodo (None si la lista está vacía)
    self.prim = None
    # Cantidad de elementos de la lista
    self.len = 0

def insert(self, i: int, x: Any) -> None:
    """Inserta el elemento x en la posición i.
    Si la posición es inválida, imprime un error y retorna inmediatamente.
    """
    if i < 0 or i > self.len:
        print("Posición inválida")
        return
    nuevo = _Nodo(x)
    if i == 0:
        # Caso particular: insertar al principio
        nuevo.prox = self.prim
        self.prim = nuevo
    else:
        # Buscar el nodo anterior a la posición deseada
        n_ant = self.prim
        for pos in range(1, i):
            n_ant = n_ant.prox
        # Intercalar el nuevo nodo
        nuevo.prox = n_ant.prox
        n_ant.prox = nuevo
    self.len += 1

def pop(self, i: int | None = None) -> Any:
    """Elimina el nodo de la posición i y devuelve el dato contenido.
    Si i está fuera de rango, muestra un mensaje de error y retorna
    inmediatamente. Si no se recibe la posición, elimina y devuelve
    el último elemento.
    """
    if i is None:
        i = self.len - 1
    if i < 0 or i >= self.len:
        print("Posición inválida")
        return
    if i == 0:
        # Caso particular: saltar la cabecera de la lista
        dato = self.prim.dato
        self.prim = self.prim.prox
    else:
        # Buscar los nodos en las posiciones (i-1) e (i)
        n_ant = self.prim
        n_act = n_ant.prox
        for pos in range(1, i):
            n_ant = n_act
            n_act = n_ant.prox
        # Guardar el dato y descartar el nodo
        dato = n_act.dato
        n_ant.prox = n_act.prox
    self.len -= 1
    return dato

```

```

def remove(self, x: Any) -> None:
    """Borra la primera aparición del valor x en la lista.
    Si x no está en la lista, imprime un mensaje de error y retorna
    inmediatamente.
    """
    if self.len == 0:
        print("La lista está vacía")
        return
    if self.prim.dato == x:
        # Caso particular: saltar la cabecera de la lista
        self.prim = self.prim.prox
        self.len -= 1
    else:
        # Buscar el nodo anterior al que contiene a x (n_ant)
        n_ant = self.prim
        n_act = n_ant.prox
        while n_act is not None and n_act.dato != x:
            n_ant = n_act
            n_act = n_act.prox
        if n_act is None:
            print("El valor no está en la lista.")
            return
        # Descartar el nodo
        n_ant.prox = n_act.prox
        self.len -= 1

def __len__(self):
    """Completar en el ejercicio 6."""
    pass

def __str__(self):
    """Completar en el ejercicio 7."""
    pass

# Faltan append e index (ejercicios 8 y 9)

```

Invariantes (condiciones que tienen que valer siempre, después de cada operación):

- `self.len` es la cantidad real de nodos de la lista.
- `self.prim` apunta al primer nodo, o es `None` si la lista está vacía.

Observá que `insert`, `pop` y `remove` actualizan `self.len` en todos sus caminos de ejecución: eso es lo que mantiene el primer invariante. Todo método nuevo que agregue o saque nodos debe hacer lo mismo.

Ejercicio 6

Implementá `__len__` para `ListaEnlazada`. Tiene que devolver la cantidad de elementos **sin recorrer** los nodos: usá el atributo `self.len`, que ya se actualiza en `insert`, `pop` y `remove`.

```
>>> L = ListaEnlazada()
>>> len(L)
0
>>> L.insert(0, "mate")
>>> L.insert(1, "bizcochos")
>>> len(L)
2
```

Para cerrar, respondé sin escribir código: si la clase no tuviera el atributo `self.len`, ¿cómo calcularías la cantidad de elementos? ¿Cuánto tardaría esa versión en una lista de un millón de nodos, comparada con la que acabás de escribir? Esa diferencia es lo que justifica mantener el contador actualizado en cada operación.

Ejercicio 7

Implementá `__str__` para `ListaEnlazada`. La representación tiene que verse como la de una lista de Python: elementos entre corchetes, separados por coma y espacio.

Si la lista está vacía, devolvés `[]`.

```
>>> L = ListaEnlazada()
>>> str(L)
'[]'
>>> L.insert(0, "mate")
>>> L.insert(1, "bizcochos")
>>> L.insert(2, "yerba")
>>> str(L)
'[mate, bizcochos, yerba]'
```

Es el mismo recorrido que hiciste en `ver_lista` (ejercicio 2), con dos diferencias: arrancás desde `self.prim` en lugar de un nodo recibido por parámetro, y en vez de imprimir cada dato lo vas acumulando en un string que devolvés al final. Cuidado con no poner una coma de más al final.

Variante opcional: escribí un método aparte `ver_enlaces()` que devuelva la misma lista pero con el formato `mate -> bizcochos -> yerba`. Es útil mientras depurás, porque deja a la vista que lo que tenés son nodos encadenados y no un arreglo.

Ejercicio 8

Implementá `append(self, x: Any) -> None`, que agrega el elemento `x` al **final** de la lista.

Podés apoyarte en `insert` (¿en qué posición insertás si querés agregar al final?) o recorrer hasta el último nodo. En ambos casos, actualizá `self.len` si hace falta (si usás `insert`, ya lo hace él).

```

>>> L = ListaEnlazada()
>>> L.append("café")
>>> L.append("medialunas")
>>> L.append("jugo")
>>> str(L)
'[café, medialunas, jugo]'
>>> len(L)
3

```

Ejercicio 9

Implementá `index(self, x: Any) -> int | None`, que devuelve la posición de la **primera** aparición de `x` en la lista. Las posiciones se numeran desde `0`.

Si `x` no está, imprimí `"El valor no está en la lista."` y devolvé `None` (igual que `remove` cuando no encuentra el valor, que usa `return` a secas). Por eso el tipo de retorno es `int | None` y no `int`: el método devuelve un entero solo cuando encontró el valor.

```

>>> L = ListaEnlazada()
>>> L.append("café")
>>> L.append("medialunas")
>>> L.append("jugo")
>>> L.append("medialunas")
>>> L.index("medialunas")
1
>>> L.index("jugo")
2
>>> L.index("tostado")
El valor no está en la lista.
>>> L.index("tostado") is None
El valor no está en la lista.
True

```

Bloque 2 — Recorridos que modifican la lista

Seguí trabajando sobre la misma `ListaEnlazada` (con `__len__`, `__str__`, `append` e `index` ya hechos).

Casos de prueba para todos los ejercicios de este bloque. Antes de dar uno por terminado, probalo con:

- la lista vacía
- una lista de un solo elemento
- el elemento buscado al principio (es el caso que obliga a tocar `self.prim`)
- el elemento buscado al final
- varias apariciones seguidas del elemento buscado
- ninguna aparición del elemento buscado

Los tres primeros son los que producen más errores, porque son justamente los casos donde no hay un “nodo anterior” disponible.

Ejercicio 10

Implementá `marcar_todos(self, buscado: Any, nuevo: Any) -> None`: recorre la lista **una sola vez** y reemplaza **todas** las apariciones de `buscado` por `nuevo`. No cambia la cantidad de nodos ni el orden.

Si `buscado` no aparece ninguna vez, la lista queda igual (no hace falta imprimir error).

Dominio de ejemplo: una playlist de episodios de un podcast, donde querés renombrar un título repetido.

```
>>> L = ListaEnlazada()
>>> L.append("intro")
>>> L.append("entrevista")
>>> L.append("cortina")
>>> L.append("entrevista")
>>> L.append("cierre")
>>> L.marcar_todos("entrevista", "charla")
>>> str(L)
'[intro, charla, cortina, charla, cierre]'
>>> len(L)
5
```

Como este método solo cambia el contenido de los nodos y no toca ningún `prox`, la cantidad de elementos no puede cambiar: `self.len` queda como estaba.

Ejercicio 11

Implementá `sacar_menores_que(self, limite: int) -> int`: elimina de la lista **todos** los números estrictamente menores que `limite`, recorriendo los nodos **una sola vez**, y

devuelve cuántos elementos sacó.

Asumí que la lista solo contiene enteros. Actualizá `self.len` para que el invariante se mantenga.

Con la lista del ejemplo y `limite = 5`:

```
Antes:  [3|•] -> [10|•] -> [1|•] -> [10|•] -> [7|•] -> [2|/]
Después:      [10|•] -----> [10|•] -> [7|/]
```

```
>>> L = ListaEnlazada()
>>> for n in (3, 10, 1, 10, 7, 2):
...     L.append(n)
>>> sacados = L.sacar_menores_que(5)
>>> sacados
3
>>> str(L)
'[10, 10, 7]'
>>> len(L)
3
```

Cuidado con el caso en que el elemento a sacar esté al principio (hay que actualizar `self.prim`) y con varios menores seguidos.

Ejercicio 12

Los dos métodos de abajo están mal implementados: **cada uno contiene más de un error**. Para cada método:

1. Identificá los errores y explicá por qué están mal.
2. Indicá cuál de los dos invariantes de la clase se rompe en cada caso (o si el error es de otro tipo).
3. Escribí una versión corregida.

Resuelvelo primero leyendo el código y siguiendo el recorrido en papel, antes de ejecutarlo.

```
def sacar_repetidos_consecutivos(self) -> None:
    """Saca los elementos que sean iguales al que tienen justo antes.
    Por ejemplo, [1, 1, 1, 2, 3, 3] debería quedar [1, 2, 3].
    """
    n_act = self.prim
    while n_act is not None:
        if n_act.prox is not None and n_act.dato == n_act.prox.dato:
            n_act.prox = n_act.prox.prox
        n_act = n_act.prox

def vaciar_hasta(self, x: Any) -> None:
    """Elimina todos los elementos anteriores a la primera aparición de x,
    de modo que x quede primero en la lista.
    """
```

```
n_act = self.prim
while n_act.dato != x:
    n_act = n_act.prox
self.prim = n_act
```

Pistas: para el primero, seguí el recorrido con la lista `[1, 1, 1]` anotando en cada vuelta a qué nodo apunta `n_act`. Para el segundo, pensá qué pasa si `x` no está en la lista, y qué pasa si la lista está vacía.

Ejercicio 13

Implementá `concatenar(self, otra: "ListaEnlazada") -> None`: agrega al final de `self` todos los elementos de `otra`, en el mismo orden. La lista `otra` no se modifica.

```
>>> compras = ListaEnlazada()
>>> compras.append("pan")
>>> compras.append("leche")
>>> extras = ListaEnlazada()
>>> extras.append("huevos")
>>> extras.append("aceite")
>>> compras.concatenar(extras)
>>> str(compras)
'[pan, leche, huevos, aceite]'
>>> str(extras)
'[huevos, aceite]'
>>> len(compras)
4
```

¿Podés estimar la complejidad de `concatenar` en función de la longitud de `self` y de `otra`, si implementás el agregado al final recorriendo desde `self.prim` cada vez (por ejemplo, llamando a `append` por cada elemento de `otra`)?

Bloque 3 — Transformaciones e inversión

Ejercicio 14

Implementá `dar_vuelta(self) -> None`, que invierte el orden de la lista **in-place**: el primer elemento pasa a ser el último y viceversa. No hace falta crear una lista nueva.

Antes: `prim -> [inicio|•] -> [desarrollo|•] -> [cierre|/]`

Después: `prim -> [cierre|•] -> [desarrollo|•] -> [inicio|/]`

```
>>> L = ListaEnlazada()
>>> for p in ("inicio", "desarrollo", "cierre"):
...     L.append(p)
>>> L.dar_vuelta()
>>> str(L)
'[cierre, desarrollo, inicio]'
>>> len(L)
3
```

Hint: andá “dando vuelta” los `prox` mientras recorrés, guardando el nodo anterior. Al terminar, `self.prim` tiene que apuntar al que antes era el último.

Tené en cuenta un detalle importante: si modificás el `prox` de un nodo antes de haber guardado a quién apuntaba, se pierde el acceso al resto de la lista y no hay forma de recuperarlo. Por eso vas a necesitar tres referencias simultáneas: el nodo anterior, el actual y el siguiente.

Ejercicio 15

Implementá `intercalar(self, valor: Any, cada_k: int) -> None`: inserta `valor` cada `cada_k` elementos de la lista original, **después** de cada bloque de `cada_k` datos.

Reglas:

- Si `cada_k <= 0`, imprimí `"k inválido"` y no modifiques la lista.
- Si la lista está vacía, no insertás nada.
- Contás solo los elementos que **ya estaban** antes de empezar; no cuentes los `valor` que vas insertando para decidir el próximo corte.
- Si la cantidad de elementos originales no es múltiplo de `cada_k`, al final queda un bloque más corto **sin** un `valor` colgando después (solo insertás cuando completaste un bloque de exactamente `cada_k`).

Conviene pensarlo por bloques. Con la lista `1 2 3 4 5 6` y `cada_k = 2`:

```
bloques de 2:  (1 2)  (3 4)  (5 6)
resultado:      1 2 0   3 4 0   5 6 0
```

Y con la lista `a b c d e` y `cada_k = 3`, el último bloque queda incompleto y por eso no lleva separador:

```
bloques de 3:  (a b c)  (d e)
resultado:      a b c -  d e
```

```
>>> L = ListaEnlazada()
>>> for x in (1, 2, 3, 4, 5, 6):
...     L.append(x)
>>> L.intercalar(0, 2)
>>> str(L)
'[1, 2, 0, 3, 4, 0, 5, 6, 0]'

>>> M = ListaEnlazada()
>>> for x in ("a", "b", "c", "d", "e"):
...     M.append(x)
>>> M.intercalar("-", 3)
>>> str(M)
'[a, b, c, -, d, e]'

>>> N = ListaEnlazada()
>>> N.append(9)
>>> N.intercalar(0, 0)
k inválido
>>> str(N)
'[9]'
```

Ejercicio 16

Escribí la función `copiar_desde(lista: ListaEnlazada, objetivo: Any) -> ListaEnlazada` que recibe una lista enlazada y un valor `objetivo`, y devuelve una **nueva** lista enlazada con todos los elementos desde la **primera** aparición de `objetivo` hasta el final (incluido el `objetivo`).

Reglas:

- La lista original no se modifica.
- Si `objetivo` no está, imprimí `"El valor no está en la lista."` y devolvés una lista enlazada vacía.
- Construí la lista nueva recorriendo desde esa posición; podés usar `append` sobre la lista nueva.

Dominio de ejemplo: historial de pestañas de un navegador; querés una copia desde cierta página en adelante.

Una aclaración sobre la forma: aunque `copiar_desde` es una función y no un método, no es código cliente. Para recorrer la lista recibida desde una posición dada necesita acceder a

`prim` y a los enlaces, porque la interfaz pública no ofrece lectura por índice: `index` te dice dónde empezar, pero no hay ninguna operación que te permita leer los elementos que siguen sin sacarlos. Por eso esta función trabaja del lado del proveedor, igual que los métodos, aun estando escrita por fuera de la clase. En el bloque 6 vas a escribir funciones que sí son clientes, y ahí la diferencia se va a ver con claridad.

```
>>> historial = ListaEnlazada()
>>> for p in ("inicio", "busqueda", "articulo", "fotos", "checkout"):
...     historial.append(p)
>>> desde = copiar_desde(historial, "articulo")
>>> str(desde)
'[articulo, fotos, checkout]'
>>> str(historial)
'[inicio, busqueda, articulo, fotos, checkout]'

>>> vacia = copiar_desde(historial, "carrito")
El valor no está en la lista.
>>> str(vacia)
'[]'
```

Bloque 4 — Lista enlazada con referencia al final

Ejercicio 17

Una desventaja de `ListaEnlazada` es que llegar al final exige recorrer todos los nodos. Eso se mitiga guardando, además de `prim`, una referencia al último nodo: `last`.

Invariante: `last` es `None` si la lista está vacía. Si no está vacía, `last` apunta al último nodo.

```
prim                                last
|                                  |
v                                  v
[turno-1|•] -> [turno-2|•] -> [turno-3|/]
```

Los nodos siguen teniendo solo `dato` y `prox`, es decir, un único enlace y hacia adelante. Lo que se duplicó son las referencias de la lista a sus extremos: `prim` y `last`.

Implementá la clase `ListaConFinal` con:

- `__init__`: lista vacía (`prim is None`, `last is None`, `len == 0`)
- `append(x)`: agrega al final en tiempo constante respecto de la longitud (actualizá `prim` si la lista estaba vacía, y siempre `last` y `len`)
- `pop(i=None)`: igual criterio que en `ListaEnlazada` (sin `i`, saca el último). Si sacás el último, actualizá `last`. Si la lista queda vacía, `prim` y `last` tienen que ser `None`. Si la posición es inválida, imprimí `"Posición inválida"` y retorná.

```
>>> D = ListaConFinal()
>>> D.append("turno-1")
>>> D.append("turno-2")
>>> D.append("turno-3")
>>> D.last.dato
'turno-3'
>>> D.pop()
'turno-3'
>>> D.last.dato
'turno-2'
>>> D.pop(0)
'turno-1'
>>> D.prim.dato
'turno-2'
>>> D.last.dato
'turno-2'
```

Para cerrar, respondé:

1. ¿Por qué `append` en esta estructura puede ser de tiempo constante, mientras que un `append` en `ListaEnlazada` que solo tiene `prim` es de tiempo lineal en la cantidad de

elementos? Explicá la diferencia en una o dos oraciones (sin apelar solo a “por definición”: mencioná qué referencia hace falta tocar y qué recorrido se evita).

2. Aun teniendo `last`, `pop()` sin índice (sacar el último) **sigue** costando tiempo lineal. ¿Por qué? Pensá qué nodo tiene que quedar apuntado por `last` después de sacar el último, y cómo hacés para llegar a él si los nodos solo tienen enlace hacia adelante. Guardá tu respuesta: esta limitación es el motivo por el que existen las listas doblemente enlazadas en sentido estricto, y en el bloque 5 vas a volver sobre ella con el nodo que la resuelve.

Bloque 5 — Listas doblemente enlazadas

La `ListaConFinal` del bloque 4 se quedó a mitad de camino. `append` dejó de recorrer la lista, pero `pop()` sin índice cuesta lo mismo que antes: después de sacar el último nodo, `last` tiene que quedar apuntando al anteúltimo, y a ese nodo no hay forma de llegar desde el final. El problema no está en la lista sino en el nodo: mientras el único enlace apunte hacia adelante, cualquier operación que necesite el anterior tiene que arrancar de `prim` y recorrer.

En este bloque cambia el nodo. Además de `prox`, cada nodo guarda `ant`, una referencia al nodo anterior:

```
class _NodoDoble:
    def __init__(self, dato: Any = None, ant=None, prox=None):
        self.dato = dato
        self.ant = ant
        self.prox = prox
```

Las cajas de los diagramas pasan a tener tres partes: el enlace hacia atrás, el dato y el enlace hacia adelante. Las flechas van en los dos sentidos y `/` sigue representando `None`:

```
      prim                                last
      |                                  |
      v                                  v
[ / | turno-1 | • ] <- -> [ • | turno-2 | • ] <- -> [ • | turno-3 | / ]
```

La lista pasa a tener tres invariantes, y el tercero es el que da trabajo:

- `prim` es `None` si la lista está vacía. Si no, `prim` es el primer nodo y `prim.ant` es `None`.
- `last` es `None` si la lista está vacía. Si no, `last` es el último nodo y `last.prox` es `None`.
- Para cualquier par de nodos vecinos, los dos enlaces tienen que coincidir: si `x.prox` es `y`, entonces `y.ant` tiene que ser `x`.

Ese tercer invariante es la diferencia real con todo lo que venías haciendo. Hasta acá, enganchar dos nodos era escribir una referencia; de ahora en más son dos, y si escribís una sola, la lista queda recorrible en un sentido y rota en el otro, sin ningún error inmediato que te avise.

Para que veas el patrón en el caso más fácil, el de agregar en un extremo, tomá `__init__`, `__len__`, `__str__` y `append` ya escritos:

```
class ListaDoblementeEnlazada:
    def __init__(self) -> None:
        self.prim = None
        self.last = None
        self.len = 0

    def __len__(self) -> int:
        return self.len
```

```

def __str__(self) -> str:
    cadena = "["
    n_act = self.prim
    while n_act is not None:
        cadena = cadena + str(n_act.dato)
        if n_act.prox is not None:
            cadena = cadena + ", "
        n_act = n_act.prox
    return cadena + "]"

def append(self, x: Any) -> None:
    # El nodo nuevo nace apuntando hacia atrás al último actual.
    nuevo = _NodoDoble(x, self.last, None)
    if self.prim is None:
        self.prim = nuevo
    else:
        self.last.prox = nuevo
    self.last = nuevo
    self.len += 1

```

Fijate `append`: el nodo nuevo nace con su `ant` apuntando al que hasta ese momento era el último, y ese último recibe un `prox` que apunta al nuevo. Dos escrituras, una por cada sentido del enlace. Los dos ejercicios que siguen son ese mismo cuidado en situaciones cada vez más incómodas.

Ejercicio 18

Agregale a `ListaDoblementeEnlazada` dos métodos que trabajan con los enlaces nuevos: uno que solo los lee y otro que los modifica.

1. `ver_al_reves(self) -> str`: devuelve los datos del último al primero, con el mismo formato que `__str__`. Arrancá en `last` y avanzá con `ant`. Es el recorrido de siempre, en el otro sentido.
2. `pop(self) -> Any`: saca el último elemento y devuelve su dato, **en tiempo constante**. Si la lista está vacía, imprimí `"Posición inválida"` y retorná. Dejá los tres invariantes valiendo: `last` pasa a ser el anteúltimo, que ahora conseguís en un paso con `last.ant`; ese nodo tiene que quedar con `prox` en `None`; y si la lista queda vacía, `prim` y `last` tienen que ser `None`. En el ejercicio 19 le vas a agregar el parámetro `i`.

```

>>> D = ListaDoblementeEnlazada()
>>> for t in ("turno-1", "turno-2", "turno-3"):
...     D.append(t)
>>> str(D)
'[turno-1, turno-2, turno-3]'
>>> D.ver_al_reves()
'[turno-3, turno-2, turno-1]'
>>> D.pop()
'turno-3'
>>> D.last.dato
'turno-2'
>>> D.last.prox is None

```

```

True
>>> str(D)
'[turno-1, turno-2]'
>>> D.pop()
'turno-2'
>>> D.pop()
'turno-1'
>>> D.prim is None and D.last is None
True
>>> D.pop()
Posición inválida

```

Para cerrar, respondé: `ListaConFinal` también tiene `last`, así que también sabe dónde termina la lista. ¿Por qué no puede hacer **este recorrido**, el de ir del último al primero siguiendo enlaces? Ojo con la trampa: el string se puede conseguir igual, recorriendo hacia adelante y armándolo al revés. Lo que la pregunta pide es por qué no se puede recorrer en ese sentido.

Ejercicio 19

Completá la clase con las dos operaciones que tocan el medio de la lista, que es donde el invariante de los enlaces cruzados se pone exigente: hay cuatro referencias en juego a la vez.

Al sacar un nodo del medio, sus dos vecinos tienen que quedar enganchados entre sí, en los dos sentidos:

```

antes de sacar turno-2

[•|turno-1|•] <--> [•|turno-2|•] <--> [•|turno-3|•]

después

[•|turno-1|•] <-----> [•|turno-3|•]

```

1. `insert(self, i: int, x: Any) -> None`: misma regla de posiciones inválidas que siempre (`i < 0` o `i > len` → "Posición inválida"). Cubrí los tres casos: al principio (`i == 0`), al final (`i == self.len`) y en el medio. Cuidá `prim`, `last` y los dos enlaces de cada vecino.
2. `pop(self, i: int | None = None) -> Any`: agregale el parámetro al `pop` del ejercicio 18. Sin `i` saca el último, igual que antes. Con `i`, saca el nodo de esa posición y deja a sus vecinos enganchados entre sí; con índice inválido, "Posición inválida".

Un control barato para saber si te quedó bien: `ver_al_reves()` tiene que dar siempre exactamente lo mismo que `str()` leído al revés. Si no coinciden, hay algún `ant` que no actualizaste, y el recorrido hacia adelante no te lo va a mostrar nunca.

```

>>> D = ListaDoblementeEnlazada()
>>> D.insert(0, "A")
>>> D.insert(1, "C")
>>> D.insert(1, "B")
>>> str(D)

```

```
'[A, B, C]'  
>>> D.ver_al_reves()  
'[C, B, A]'  
>>> D.pop(1)  
'B'  
>>> str(D)  
'[A, C]'  
>>> D.ver_al_reves()  
'[C, A]'  
>>> D.pop()  
'C'  
>>> D.last.dato  
'A'  
>>> D.insert(5, "Z")  
Posición inválida  
>>> D.pop(9)  
Posición inválida  
>>> str(D)  
'[A]'
```

Para cerrar, volvé a leer lo que respondiste en la segunda pregunta del ejercicio 17 y respondé ahora: ¿cuánto cuesta `pop()` sin índice en esta clase? ¿Y `pop(i)` con `i` en el medio de la lista? ¿Qué parte del trabajo abarató el enlace `ant` y qué parte quedó igual?

Bloque 6 — Usar el TAD desde afuera

Hasta acá trabajaste siempre del lado del **proveedor**, incluso en el ejercicio 16, donde escribiste una función por fuera de la clase: en todos los casos accediste a `prim` y a los enlaces. En este bloque cambiás de rol y trabajás como **cliente**: resolvés un problema usando únicamente las operaciones que el TAD ofrece, sin enterarte de cómo está implementado por dentro.

Ejercicio 20

Una guardia de consultorio maneja dos listas: los pacientes que esperan y los que ya fueron atendidos.

Escribí estas dos funciones, que reciben listas enlazadas y las modifican:

- `atender_siguiente(espera: ListaEnlazada, atendidos: ListaEnlazada) -> None` : saca al primer paciente de `espera` y lo agrega al final de `atendidos` . Si `espera` está vacía, imprime `"No hay pacientes en espera"` y no modifica nada.
- `adelantar(espera: ListaEnlazada, paciente: Any) -> None` : mueve a `paciente` al principio de `espera` . Si el paciente no está en `espera` , no hace falta que imprimas nada extra: alcanza con el mensaje que ya emite `index` .

Restricción: dentro de estas funciones solo podés usar `append` , `insert` , `pop` , `remove` , `index` , `len()` y `str()` . No podés acceder a `.prim` , ni a `.prox` , ni crear nodos con `_Nodo` .

```
>>> espera = ListaEnlazada()
>>> for p in ("Rocío", "Tomás", "Valentina"):
...     espera.append(p)
>>> atendidos = ListaEnlazada()
>>> atender_siguiente(espera, atendidos)
>>> str(espera)
'[Tomás, Valentina]'
>>> str(atendidos)
'[Rocío]'
>>> adelantar(espera, "Valentina")
>>> str(espera)
'[Valentina, Tomás]'
>>> vacia = ListaEnlazada()
>>> atender_siguiente(vacia, atendidos)
No hay pacientes en espera
```

Para cerrar, respondé: si mañana reemplazaras `ListaEnlazada` por otra implementación del mismo TAD, que ofrezca las mismas operaciones pero guarde los datos de otra manera (por ejemplo la `ListaDoblementeEnlazada` del bloque 5, con sus enlaces en los dos sentidos), ¿habría que cambiar algo de estas dos funciones? ¿Y qué pasaría si las hubieras escrito accediendo directamente a `.prim` y a `.prox` ?